Mashup-Oriented Web API Recommendation Via Full-Text Semantic Mining of Developer Requirements

Chunyan Sang, Xinyan Deng, and Shigen Liao

***Abstract*—API recommendation differs from conventional recommendation systems in that it integrates the characteristics of development requirements, mashup, and API, where there are two primary challenges, i.e., how to effectively mine the personalized development requirements of developers and the sparseness of interaction data. This research proposes a novel model framework, called Bayesian Probabilistic Matrix Factorization Model with Text Similarity and Adversarial Training (SAMF), to address these two issues. Utilizing natural language processing technology to extract the full-text semantics of requirements documents, mashup descriptions, and API descriptions, fully mining personalized development requirements, and contextual information of mashups and APIs, and calculating text similarity, is our fundamental idea. Simultaneously, the collaborative filtering method is used to mine user needs, mashup, and API information from historical data, and the obtained text similarity is added to the training as auxiliary information. Furthermore, adversarial training is further incorporated to supplement data in order to minimize data sparsity and enhance model robustness and generalization. To assess the performance of SAMF, we run thorough experiments that illustrate the efficacy of each module of the model and explain the influence of hyperparameter settings. Specifically, compared to the baseline, the experimental findings demonstrate that SAMF can basically achieve better performance.**

***Index Terms*—API recommendations, adversarial training, full-text semantics, personalize, robust.**

I. Introduction

WEB services (or Web APIs) have grown in quantity as the field of service-oriented computing has continued to expand in recent years [1], [2]. The complexity of the present business scenario is not as straightforward as it once was; thus, the existing single API no longer meets our requirements. In such an environment, mashups, which typically serve context-specific (ephemeral) needs and consist of the most up-to-date, simple-to-use APIs, have become an important technology for integrating multiple services to address complicated needs [3], [4], [5]. Due to several advantages, mashups and APIs have also been extensively developed and used in web and mobile apps [6], [7] in recent years. Given the vast number of APIs and their incredibly rapid growth rate, it is challenging for mashup developers to find APIs that cover all needs in a short amount of time, which undoubtedly increases the difficulty of mashup development. API recommendation is based on the functional needs of mashup developers in order to propose a selection of relevant APIs from the enormous number of APIs for recommendation, and the general process is shown in Fig. 1. A developer wants to develop a mashup and writes a requirements document after performing a requirements analysis, then splits the requirements into several features. In order to find suitable APIs to implement these functions, the developer needs to search for suitable APIs with the required functions in an online mashups-APIs storage platform. And select the APIs corresponding to the functions from these candidate APIs to combine the functions required for the mashup. In recent years, business and academics have paid more attention to API recommendation [8] owing to the fact that it is vital for developers to effectively find suitable APIs or services in application development.

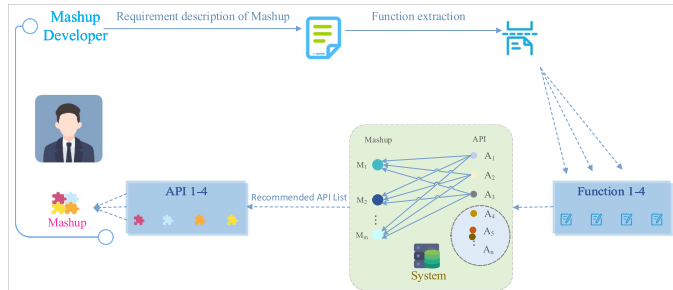

Fig. 1. An example of API recommendation.

API recommendations always want to provide developers with APIs that meet their needs [9], [10], [11] and have a high quality of service (i.e., QoS) [12], [13], [14]. In contrast to QoS-based API recommendations, developers need more APIs that are highly suited to individual requirements for building mashups [15], [16]. On the platform for sharing web APIs, each API usually has multiple tags or keywords to represent its functionalities. Then, the content-based API recommendation strategy may be implemented, in which text mining techniques

Manuscript received 18 July 2022; revised 19 January 2023; accepted 13 February 2023. Date of publication 16 February 2023; date of current version 8 August 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62002037, and in part by the research project of Chongqing CSTC under Grant cstc2019jcyj-msxmX0588. Recommended for acceptance by B. Benatallah. *(Corresponding author: Chunyan Sang.)*

The authors are with the School of Software Engineering, ChongQing University of Posts and Telecommunications, Chongqing 400065, China (e-mail: sangcy@cqupt.edu.cn; s211231010@stu.cqupt.edu.cn; liaosg@cqupt.edu.cn).

Digital Object Identifier 10.1109/TSC.2023.3245652

1939-1374 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:02:02 UTC from IEEE Xplore. Restrictions apply.

TABLE I
RECOMMENDED APPROACHES

Methods	Representation type of requirement	Aspects to be considered	Advantage	Shortcoming
WARtext [20]	Keywords	Developer, Mashup, and API	Focus on the needs of developers	Restricting keywords description requirements cannot maintain semantic relationships like documents
AMF+ [21]	Documents	Mashup and API	Allows documents to describe requirements, maintaining semantic relationships	Ignore developer needs
Ours	Documents	Developer, Mashup, and API	Focus on the needs of developers	

are often used to mine information between user requirements and API functions, or between user requirements and mashup functions. Including converting the keywords required by the user into regular expressions to interact with the configuration file [17]; using LDA to calculate the similarity of topic keywords for Web API and user requirements [18]; and using LDA to calculate the similarity of topic keywords between user requirements and mashup [19]; according to TF-IDF from the sentence block of mashup requirements and the semantics of mashup description similarity [15]. These proposed methods crawl keywords for API recommendation through information from two aspects (i.e., user and API or user and mashup).

Almost no relevant tags exist in the real world that can be used to effectively integrate APIs; therefore, it is insufficient to describe complicated development requirements with only a few keywords, nor can it fully express the individual needs of developers [14], [19], [20]. Some studies have begun to shift their attention from keyword descriptions to textual ones. Ref. [21] stored the developer's API query input (consisting of a set of phrases, sentences, or even paragraphs) into a collection, as well as the existing keywords for each API, transforming them to vectors using FastText technology. Calculating the similarity of keywords yielded a set of candidate APIs, which were then compared to the APIs in the existing mashup to provide the final list of recommendations. The research includes suggestions from three viewpoints (developer, mashup, and API), which are more representative of actual application development situations. However, without expert understanding, mashup developers are unable to describe their development requirements using keywords, refined sentences, or paragraphs. Requirements documents can clearly indicate their individual requirements. The state of the art research [22] utilized Doc2Vec technology to mine mashup and API description information, calculated the text similarity between the two, and then generated recommendations based on the similarity and the API information in the existing mashup. This type of natural language processing technology for processing long texts may effectively mine text information as it extracts the whole event instead of the key words and does not destroy the semantic relationship between sentences. The fly in the ointment is that it is impossible to obtain more personalized needs of developers by mining information only from the two aspects of mashup and API. We intend to build mashup-oriented API recommendations from user requirements, mashup descriptions, and API descriptions, utilizing natural language processing technology processing techniques to extract functional description information and semantic information from long texts. Table I will more intuitively demonstrate the intuition we need to solve the problems found in the literature.

In order to mine user needs, mashups, and API information from historical data and estimate the probability of users choosing APIs, many existing recommendation systems use the Collaborative Filtering (CF) method, which is also a widely used technique in API recommendation. CF is a traditional and popular method of recommendation, which can be roughly divided into two categories: one is memory-based [23], [24], [25], and the other is model-based [26], [27], [28], [29], [30]. The Probabilistic Matrix Factorization (PMF) [31] is one of the popular models of the latter type, which has lower space requirements and faster training and prediction speeds. Although there are numerous recommended methods based on PMF models [22], [32], [33], [34], [35], they still have some deficiencies, such as this method's susceptibility to overfitting if the regularization parameter is not adjusted extremely carefully, as it only discovers a single point of the parameter estimate. Nonetheless, Bayesian Probability Matrix Factorization (BPMF) [36] automatically controls the model expression space by integrating all model parameters and hyperparameters, resulting in high prediction accuracy and good scalability, and is frequently employed in personalized recommendation systems [37]. In addition, the fast expansion of APIs makes invocation data of mashups and APIs sparser, as shown in the green frame in Fig. 1, as just a few APIs are frequently invoked and most APIs are not even invoked by mashups, which affects the accuracy of model predictions [38]. Frequently, many models use context information to address this issue, such as [39] introduces context nodes. In addition, the use of context to mine auxiliary information and add it to model training can also alleviate the data sparsity problem; for instance, [40], [41], [42], [43] utilizes auxiliary information such as spatiotemporal information, description, comment, and credibility, respectively. For this purpose, we adopt a BPMF-based recommendation method, fuse the obtained context information as auxiliary information, and introduce adversarial training to improve recommendation accuracy and alleviate data sparsity.

Inspired by the above, this paper proposes a new model framework named Bayesian Probabilistic Matrix Factorization Models with Text Similarity and Adversarial Training (henceforth referred to as SAMF), which can achieve high-precision API recommendation outcomes with little computing cost. Specifically, we use Doc2Vec, a natural language processing technology that processes long texts, to mine context information from three aspects (requirements document, mashup description, and API description) in order to assist mashup developers without professional expertise in describing the development requirements

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:02:02 UTC from IEEE Xplore. Restrictions apply.

of mashups for the purpose of personalized recommendation. Instead of scraping keywords, we directly extract events in documents to preserve semantic relationships to the greatest extent and represent personalized needs more accurately. At the same time, to address the issue of sparser mashup and API invocation data, this work leverages contextual information as auxiliary information to enhance the performance of the model by incorporating text similarity into training. In addition, adversarial training is included in the model by introducing noise to the current data in order to combat data sparsity and increase the model's robustness, resulting in a more generic model. The main contributions of this work are as follows:

- We propose a new model called SAMF, which takes into account recommendations from three aspects (mashup developer, mashup, and API), and can capture the more personalized needs of developers.
- In order to meet the developer's demands to the maximum extent possible, we permit mashup developers to express their particular development needs using documents, making it easier for mashup developers without professional expertise to describe their needs. We use Doc2Vec technology to mine requirement documents, mashup descriptions, and API descriptions by extracting the entire event rather than grabbing keywords to mine full-text semantics. This ensures that the mined information maintains the semantic relationship to the greatest extent possible and more accurately reflects the individual needs of developers.
- To overcome the issue of sparse data, we use the obtained context information as auxiliary information in the model and introduce adversarial training to add noise to the data in order to further solve the problem as well as improve the model's generalization robustness.
- We separated the ProgrammableWeb dataset into several training sets with different percentages of masking, and performed extensive experiments with them. In addition to the comparison with common methods, it is also compared with the latest method AMF+[22], which uses the idea of matrix decomposition to recommend APIs. In order to verify that document input is more efficient than keyword input, FastText adopted in WARtext [21] is used by us to extract the requirement keywords as input to replace the document input in our SAMF model. The results showed that our model substantially outperformed all baseline models.

The remaining sections of this work are structured as follows. Section II summarizes related work. Section III describes the recommended methodology. Section IV describes the experimental design, implementation, results, and discussion. The research is concluded in Section V.

II. Related Work

A. Content-Based API Recommendations

Web APIs are similar to web services, in terms of input data for web service recommendation, not only semantic data but also explicit feedback such as rating data (e.g., QoS) of web services is often used [44]. Zheng et al. [45] and Shao et al. [46] recommended APIs by comparing the QoS scores of mashups. Fan et al. [2] extended the traditional CF technique by utilizing QoS data based on spatiotemporal validity. These studies concentrate on suggesting APIs or services with appropriate QoS (Quality of Service). In addition to focusing on the information about the API itself, more research has begun to focus on the information between user needs and APIs. Zhang et al. [18] represented both web APIs and user requirements as word sets, then used LDA to model topics to extract topic model keywords, and finally calculated the similarity between two topics for recommendation. This approach avoided complex semantic annotations while capturing the dependency relationship between words describing the hidden topics of the service and the functionally representative features of the service. Cao et al. [15] divided the requirement description of mashup into sentence blocks, extracted keywords from sentence blocks and existing mashup descriptions according to TF-IDF, and then calculated the semantic similarity between keywords for recommendation. By mining existing mashups and their descriptions using discourse analysis, the method revealed semantic relationships between services. Cao et al. [19] used LDA text mining technology to determine the similarity between user needs and mashups, and then recommended APIs from the most comparable mashups to users. This approach created a domain-aware, fine-grained selection and reduction procedure for mashup service features in order to get reduced description texts. Xia et al. [9] suggested clustering services based on categories and using collaborative filtering to forecast the ranking order of services within each category after modeling service descriptions using topic models. In order to increase the precision of service clustering, Cao et al. [5] developed a two-level topic model exploiting the connection between mashups and services and mined potentially valuable topics. To take advantage of mashup descriptions and structure and close the lexical gap between mashup developers and Web APIs, Zhong et al. [43] suggested modeling mashup descriptions and Web APIs together using an author-topic model.

B. Collaborative-Based API Recommendations

With the advancement of machine learning, the use of Collaborative Filtering (CF) in web services has become widespread. Memory-based and model-based methods are two main forms of CF. Memory-based CF directly uses the statistical information in the existing behavior matrix to make predictions, while model-based collaborative filtering mainly extracts feature information from data and uses matrix factorization technology to make recommendations. Therefore, both of the above two methods can utilize the interaction matrix of mashup and API. Yao et al. [47] mined the implicit correlations of APIs, and used the probability matrix factorization method of implicit correlation regularization for API recommendation, showing that implicit similarity can improve recommendation performance. Fletcher [48] extracted the user's implicit preference from the invocation history and incorporated it into the matrix factorization model in order to increase the recommendation's precision.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:02:02 UTC from IEEE Xplore. Restrictions apply.

With the fast advancement of deep learning in recent years, deep neural networks have been used to enhance MF. NeuMF [49] used Multi-Layer Perceptron (MLP) to capture nonlinear features of user-item interactions. AFM+ [22] utilized MLP as an attention mechanism to learn attention scores (i.e., the weights of feature interactions) that indicate the significance of interactions in predictions. In addition, there are graphs or network models to represent mashup and API data and recommend APIs based on that. For instance, WARtext [21] introduced a naive weighted web API correlation graph and then used Steiner trees to determine the optimal API recommendation list. This approach specifically mined past combined usage of APIs to model API compatibility and guaranteed to return the most compatible set of APIs. Based on the Mashup-API co-invocation patterns and service category properties, Wang et al. [6] constructed an improved knowledge graph and provided API suggestions. It dynamically changed entity preferences that correspond to distinct equivalences (i.e., API-based neighborhoods or Mashup-based neighborhoods) using an entity-biased random walk technique, hence enhancing the precision of API recommendations.

III. Proposed Method

In this section, the issue of API recommendation will be briefly discussed, followed by a detailed explanation of the model approach.

A. Problem Description

Given the past invocation of APIs in prior mashups, we define API recommendation as recommending an appropriate collection of APIs for the target mashup. Simply said, an API recommended to a mashup should have a high probability of being followed by that mashup. The probability that the mashup will follow APIs that are not recommended for the mashup is reduced. Assume that there are invocation records of n APIs in m mashups. Both the mashup-specific and API-specific latent feature vectors are d-dimensional, and they are denoted by matrix M and A, respectively, where $M \in R^{d\times m}$, $A \in R^{d\times n}$. Element R_{ij} of matrix $R \in R^{m\times n}$ of mashup and API invocation history records represents whether API A_j has been invoked by mashup M_i ($R_{ij} = 1$ if invoked, otherwise 0). Then, the probability $\hat{R_{ij}}$ that mashup M_i invoke API A_j is given in (1).

$$\hat{R_{ij}} = S_{ij} M_i^T A_j \tag{1}$$

Unlike BPMF, which defines $\hat{R_{ij}}$ as the dot product of two latent feature vector M_i and A_j, we add a text similarity S_{ij} (which will be introduced in Section III-B1) in the process of predicting $\hat{R_{ij}}$, to emphasize the importance of developing requirements.

B. Architectural Framework

The SAMF is depicted in Fig. 2; particularly, we discuss a Bayesian probabilistic matrix factorization-based method for making recommendations. In particular, SAMF is composed of three components. In Step 1, SAMF employs Doc2Vec technology to directly extract the full-text semantics of the mashup

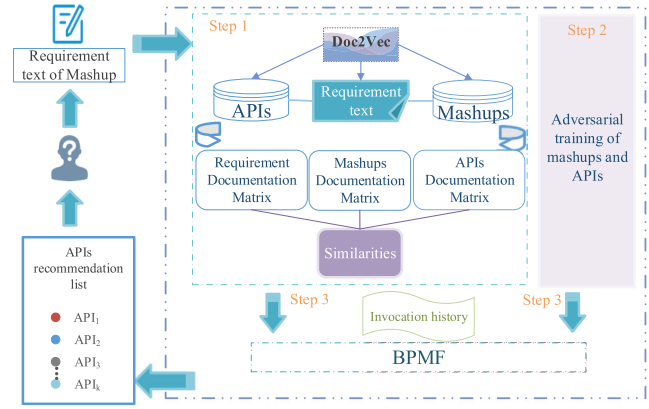

Fig. 2. The Framework of SAMF.

developer's requirements documents, mashups, and APIs, vectorizes them, and then calculates their text similarity. In addition, we implemented adversarial training in Step 2 to address the robustness and generalizability of the model and alleviate the problem of data sparsity. After obtaining textual similarity and adversarial samples, we proceed to Step 3, integrating the BPMF method to produce a ranked list of relevant APIs as a recommendation result. Details of Step 1 to Step 3 are described in Sections III-B1, III-B2, and III-B3.

1) Text Similarity: The quantitative representation of text is a difficult topic in machine learning, but the Doc2Vec technology has effectively and universally overcome this issue. In this study, we use the Doc2Vec technique to text-train the descriptions of mashups and APIs, and vectorize the description documents of both. After this step, the feature vector M_i and A_j of the required text of the mashup and API respectively are extracted by the Doc2Vec model. Finally, the text similarity $S_{ij} \in R^{m\times n}$ between M_i and A_j is calculated, as shown in (2). Furthermore, we add S_{ij} as a regularization term into the prediction. Here we choose to use the cosine formula to calculate the similarity of text, which is a common method in natural language processing [50], [51].

$$S_{ij} = \frac{M_i \bullet A_j}{\|M_i\| \times \|A_j\|} \tag{2}$$

At this time, the calculation method of the probability $\hat{R_{ij}}$ is as shown in (1). Further, we use the idea of probabilistic matrix decomposition to obtain the gradients G_{M_i} and G_{A_j} of the feature vectors of the mashup M_i and API A_j, as shown in (3), so as to update the parameters M_i and A_j subsequently, where λ_S is the regular parameter and err is the error between the real value and the predicted value ($err = R_{ij} - \hat{R_{ij}} = R_{ij} - S_{ij} M_i^T A_j$).

$$\begin{cases} G_{M_i} = A_j err^T - \lambda_S M_i - \lambda_S A_j \hat{R_{ij}}^T \\ G_{A_j} = M_i err - \lambda_S A_j - \lambda_S M_i \hat{R_{ij}} \end{cases} \tag{3}$$

Then M_i and A_j are updated as shown in (4), where lr is the learning rate in the formula below.

$$\begin{cases} M_i = M_i + lr G_{M_i} \\ A_j = A_j + lr G_{A_j} \end{cases} \tag{4}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:02:02 UTC from IEEE Xplore. Restrictions apply.

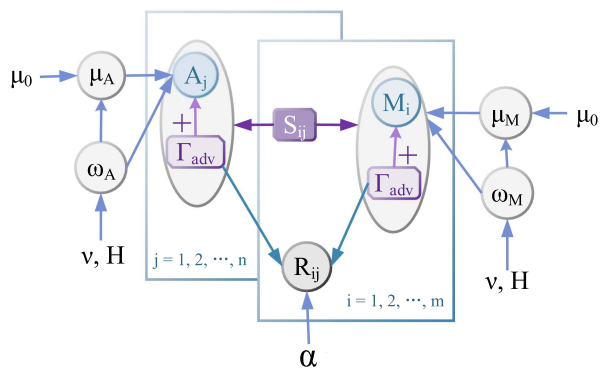

Fig. 3. Graphical model of SAMF.

2) Adversarial Example Generation: This research leverages adversarial training to increase the model's robustness and generalizability, as well as to reduce data sparsity. It should be noted that the Fast Gradient Method (FGM) is already a mature adversarial training model. Therefore, FGM is used for adversarial training in this study, as shown in (5).

$$\Gamma_{adv} = \frac{\epsilon \cdot g}{\|g\|_2} \tag{5}$$

There are two parameters in the (5), where ϵ is the parameter and g is the gradient of the embedding. Specifically, we add the perturbation factor Γ_{adv} obtained in (5) to M_i and A_j on the basis of (4) to form a new vector. The formula is shown below.

$$\begin{cases} M_i = M_i + \Gamma_{adv} \\ A_j = A_j + \Gamma_{adv} \end{cases} \tag{6}$$

3) SAMF Model: We have established text similarity and adversarial examples (refer to Sections III-B1 and III-B2). This study uses both of them to improve the SAMF model in order to create a more robust model, so the following work is carried out on the basis of (6). Fig. 3 illustrates the form of a probabilistic graphical model of SAMF, with parameters detailed in context.

Assume that the prior distribution of the mashup and API feature vector M_i and A_j is Gaussian:

$$p(M\,|\mu_M, \omega_M) = \prod_{i=1}^{m} N\left(M_i\,|\mu_M\,, \omega_M^{-1}\right), \tag{7}$$

$$p(A\,|\mu_A, \omega_A) = \prod_{j=1}^{n} N\left(A_j\,|\mu_A\,, \omega_A^{-1}\right). \tag{8}$$

Furthermore, Gaussian-Wishart priors are further placed on the mashup and API hyperparameters $\Theta_M = \{\mu_M, \omega_M\}$ and $\Theta_A = \{\mu_A, \omega_A\}$ as follows.

$$\begin{aligned} p(\Theta_M\,|\Theta_0) &= p(\mu_M\,|\omega_M)p(\omega_M) \\ &= N(\mu_M\,\Big|\mu_0, (\beta\omega_M)^{-1})f(\omega_M\,|\nu, H) \end{aligned} \tag{9}$$

$$\begin{aligned} p(\Theta_A\,|\Theta_0) &= p(\mu_A\,|\omega_A)p(\omega_A) \\ &= N(\mu_A\,\Big|\mu_0, (\beta\omega_A)^{-1})f(\omega_A\,|\nu, H) \end{aligned} \tag{10}$$

In the above formula, f is the probability density function of the Wishart distribution, in the above formula, ω is a $d \times d$ random matrix, H is a $d \times d$ specific matrix, and ν represents degrees of freedom.

$$f(\omega\,|\nu, H) = \frac{|\omega|^{(\nu-d-1)/2}}{2^{\nu d/2}\,\Gamma_d\,(\nu/2)\,|H|^{\nu/2}} \exp\left[-\frac{1}{2}tr(H^{-1}\omega)\right]. \tag{11}$$

In the above formula, $\Gamma_d(\nu/2)$ is a multivariate gamma function $\Gamma_d(\nu/2) = \pi^{d(d-1)/4}\prod_{j=1}^{d}\Gamma(\frac{\nu+1-j}{2})$, $tr(H^{-1}\omega)$ is the trace of the matrix. In addition to this, we set $\Theta_0 = \{\mu_0, \nu, H\}$, $\nu = d$, and H as identity matrices for the mashup and API hyperparameters, and choose $\mu_0 = 0$ by symmetry.

The predicted distributions of the rating values R_{ij} for mashup M_i and API A_j will be obtained by marginalizing the model parameters and hyperparameters. Due to the complexity of the posterior, accurate evaluation of such predicted distributions is difficult, and therefore, resort to approximate inference is required. The MCMC-based method [52] uses a Monte Carlo approximation to predict the distribution of the equation. The formula is as follows:

$$\begin{aligned} p({R^*}_{ij}\,|R, \Theta_0) &= \iint p({R^*}_{ij}\,|M_i, A_j)p(M, A\,|\Theta_M, \Theta_A) \\ &\quad p(\Theta_M, \Theta_A\,|\Theta_0)d\{\Theta_M, \Theta_A\} \\ &\approx \frac{1}{K}\sum_{k=1}^{K} p({R^*}_{ij}\,\Big|{M_i}^{(k)}, {A_j}^{(k)}). \end{aligned} \tag{12}$$

The sample $\{{M_i}^{(k)}, {A_j}^{(k)}\}$ in the above formula is a Markov chain generated by running, and its stationary distribution is the posterior distribution of the model parameters and hyperparameters $\{M, A, \Theta_M, \Theta_A\}$.

The BPMF model uses conjugate priors on parameters and hyperparameters, and the hyperparameter values have a Gaussian distribution as shown below. Ref. [36] has also shown that the conditional distribution derived from the posterior distribution is straightforward to sample from.

$$\begin{aligned} p\,(M_i\,|R, A, \Theta_M, \alpha) &= N\left(M_i\,\Big|{\mu_i}^*, [{\omega_i}^*]^{-1}\right) \\ &\sim \prod_{j=1}^{n}\left[N\left(R_{ij}\,|M_i^T A_j, \alpha^{-1}\right)\right]^{I_{ij}} \\ &\quad p\,(M_i\,|\mu_M, \omega_M) \end{aligned} \tag{13}$$

Among them, $\omega_i^* = \omega_M + \alpha\sum_{j=1}^{n}[A_j {A_j}^T]^{I_{ij}}$, $\mu_i^* = [\omega_i^*]^{-1}(\alpha\sum_{j=1}^{n}[A_j R_{ij}]^{I_{ij}} + \omega_M\mu_M)$.

It is worth noting that Gibbs sampling, one of the most straightforward MCMC algorithms, is often utilized when these conditional distributions can be sampled easily. The Gaussian-Wishart distribution, for instance, provides the conditional distribution of the mashup hyperparameters conditioned on the mashup feature matrix M:

$$P(\mu_M, \omega_M|M, \Theta_0) = N(\mu_M\Big|{\mu_0}^*, (\beta^*\omega_M)^{-1})f(\omega_M|\nu^*, H^*). \tag{14}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:02:02 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Gibbs Sampling for SAMF.

Input: :
The model parameters and hyperparameters $\{M, A, \Theta_M, \Theta_A\}$.
Output:
Sample Set $\{M_i{}^{(k)}, A_j{}^{(k)}\}$.
Initialize model parameters $\{M^{(-1)}, A^{(-1)}\}$;
for *each* $\rho \in T$ **do**
  sample the hyperparameters (14), (16):
    $\Theta_M^{epoch} \sim p(\Theta_M | M^{epoch}, \Theta_0)$
    $\Theta_A^{epoch} \sim p(\Theta_A | A^{epoch}, \Theta_0)$
  for *each* $i \in m$ **do**
    sample mashup features in parallel (13):
    $M_i^{epoch+1} \sim p\left(M_i | R, A^{epoch+1}, \Theta_M^{epoch}\right)$
  end
  for *each* $j \in n$ **do**
    sample API features in parallel (15):
    $A_j^{epoch+1} \sim p\left(A_j | R, M^{epoch+1}, \Theta_A^{epoch}\right)$
  end
end

The meanings of the parameters in the above formula are as follows: $\mu_0{}^* = \frac{\beta\mu_0 + m\overline{M}}{\beta+m}$, $\beta^* = \beta + m$, $\nu^* = \nu + m$, $[H^*]^{-1} = H^{-1} + m\overline{S} + \frac{\beta m}{\beta+m}(\mu_0 - \overline{M})(\mu_0 - \overline{M})^T$,
$\overline{M} = \frac{1}{m}\sum_{i=1}^{m} M_i$, $\overline{S} = \frac{1}{m}\sum_{i=1}^{m} M_i M_i{}^T$.

The conditional distribution of API hyperparameters conditioned on the API feature matrix A has exactly the same form as shown below.

$$p\left(A_j | R, M, \Theta_A, \alpha\right) = N\left(A_j \middle| \mu_j{}^*, [\omega_j{}^*]^{-1}\right) \sim \prod_{i=1}^{m} \left[N\left(R_{ij} \middle| M_i^T A_j, \alpha^{-1}\right)\right]^{I_{ij}} p\left(A_j | \mu_A, \omega_A\right) \tag{15}$$

$$P(\mu_A, \omega_A | A, \Theta_0) = N(\mu_A \left| \mu_0{}^*, (\beta^* \omega_A)^{-1}\right) f(\omega_A | \nu^*, H^*) \tag{16}$$

The details of Gibbs sampling are shown in Algorithm 1.

IV. Experiments

This section describes the experimental setup and analysis to assess the performance of our proposed method in API recommendation. The following research questions will be answered via experiments:

-RQ1 How is the performance of our proposed method compare to frequently used methods and state-of-the-art methods?

-RQ2 How do auxiliary information and adversarial training influence the performance of our proposed method?

-RQ3 How do hyperparameters affect performance, and how can optimum values be determined?

TABLE II
Statistical Analysis of the Experimental Datasets

Dataset	Interaction#	API#	Mashup#	Sparsity(%)
ProgrammableWeb	10737	1170	5691	99.8387
Train dataset #5	5368	1170	5691	99.9194
Train dataset #4	6442	1170	5691	99.9033
Train dataset #3	7515	1170	5691	99.8871
Train dataset #2	8589	1170	5691	99.8710
Train dataset #1	9663	1170	5691	99.8549
Test dataset	10737	1170	5691	99.8387

A. Experimental Settings

1) Datasets: To verify the efficacy of the proposed method, we use ProgrammableWeb's dataset. The dataset contains 17829 APIs, 6340 mashups, and their historical invocations. Table II provides a summary of the dataset's statistics (after all preprocessing steps).

The following is a brief description of data processing. Since the datasets lack ratings for mashups and APIs, we utilize their invocation data as a proxy for ratings. For instance, if mashup M_1 calls API A_9 and their invocation data is 1, then that data should be used as the rating for that mashup-API combination. Subsequently, we removed blank data and duplicate data. Finally, the original data is utilized as the test set. After masking a portion of the original data's interaction data, the training set is produced, specifically, assigning all the data in the masked position to 0. For example, Train dataset #5 in Table II masks 50% of interactions, Train dataset #4 masks 40% of interactions, and the rest masks 30%, 20%, and 10% of interactions, respectively.

2) Baselines: Currently, there is no general baseline model for recommending API for mashup. As a result, for the purpose of comparing and evaluating our proposed model, we selected certain frequently used methods and state-of-the-art method cited in related work as a baseline for comparison. Among them, PMF and BPMF are common methods, FastText-MF is adding natural language processing technology to common methods, and AMF+ is the state-of-the-art method.

-PMF [31]: PMF employs mashup and API interactions for collaborative filtering.

-BPMF [36]: Bayesian Probabilistic Matrix Factorization for collaborative filtering of interaction data for mashups and APIs, this model is an improvement over PMF, which solves problems from a Bayesian perspective rather than a traditional probabilistic perspective.

-FastText-MF: The model uses FastText [53] instead of Doc2Vec [54] technology to train requirement. FastText adopted in WARtext [21] is used by us to extract the requirement keywords. After it is vectorized, the text similarity is calculated by the cosine formula, and finally attached to the BPMF as the regularization part.

-FastText-SAMF: Using FastText adopted in WARtext [21] to extract the requirement keywords as input, to replace the document input in our SAMF model.

-AMF+ [22]: This model features context-awareness regularization and API co-invocation interactions.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:02:02 UTC from IEEE Xplore. Restrictions apply.

TABLE III
NDCG COMPARISON AT K = 10

	NDCG@10(Train dataset)				
	#1	#2	#3	#4	#5
PMF	0.2147	0.2199	0.1815	0.1961	0.1817
BPMF	0.2903	0.3036	0.3136	0.3388	0.3894
FastText-MF	0.3547	0.3712	0.3755	0.3874	0.3921
FastText-SAMF	0.3599	0.3759	0.3796	0.3922	0.3980
AMF+	0.3597	0.3748	0.3804	0.3926	0.3982
SAMF	**0.3611**	**0.3749**	**0.3798**	**0.3925**	**0.3984**

TABLE IV
NDCG COMPARISON AT K = 7

	NDCG@7(Train dataset)				
	#1	#2	#3	#4	#5
PMF	0.1934	0.1880	0.1633	0.1615	0.1649
BPMF	0.2728	0.2865	0.3000	0.3247	0.3307
FastText-MF	0.3407	0.3576	0.3642	0.3748	0.3795
FastText-SAMF	0.3483	0.3628	0.3684	0.3802	0.3863
AMF+	0.3483	0.3625	0.3680	0.3801	0.3861
SAMF	**0.3482**	**0.3630**	**0.3686**	**0.3805**	**0.3863**

3) Evaluation Protocol: To evaluate the performance of the TOP-K recommendation, we truncate the position of the recommendation list at K, with K equal to 1, 3, 5, 7, and 10. In addition, to evaluate the efficacy of the recommendation list generated by our proposed method, we adopt two commonly used metrics for recommendation: Normalized Discounted Cumulative Gain (NDCG) and Mean Average Precision (MAP), respectively. The calculation formulas for these two recommended metrics are as follows:

$$NDCG@K = \sum_{i=1}^{n} \frac{\left(2^{r(i)} - 1\right)}{\log_2(1+i)}, \tag{17}$$

$$MAP@K = \frac{1}{|m|} \sum_{i=1}^{|m|} AP@K, \tag{18}$$

$$AP@K = \frac{1}{n} \sum_{i=1}^{K} hit(i) \times \frac{1}{p_i}. \tag{19}$$

In 17, 18, and 19, n and m represent the number of APIs and mashups, respectively. The function $hit(i)$ indicates whether it is a hit, that is, whether the API selected by the mashup is in the recommended sequence, 1 if it exists, and 0 otherwise. The value of p_i indicates the position of the API accessed by the mashup in the recommendation list. If it is not in the recommendation sequence, then p is infinite, and $\frac{1}{p}$ is 0.

4) Implementation and Configuration Parameter: Our implementation is based on the PyTorch framework. To improve performance, we modify the model's hyperparameters and choose the ideal hyperparameters based on NDCG@10.

For this study, the latent dimension d is tuned in $[5, 10, 15, \ldots, 45, 50]$. For SAMF, we tune lr in $[0.000005, 0.00001, 0.00005, 0.0001, 0.0005, 0.001, 0.005, 0.01]$ and λ_S in $[1.0, 0.5, 0.1, 0.05, 0.01, 0.005, 0.001]$. SAMF achieves good performance when $d = 20$, $lr = 0.0001$ and $\lambda_S = 0.1$ on datasets.

B. Experimental Results

1) Comparison With Baselines Performance (RQ 1): To answer RQ1, we evaluate our proposed method's performance to baselines in this subsection. The specifics of the NDCG@K and MAP@K experiment results are shown in Tables III, IV, V, VI,

TABLE V
NDCG COMPARISON AT K = 5

	NDCG@5(Train dataset)				
	#1	#2	#3	#4	#5
PMF	0.1920	0.1809	0.1716	0.1736	0.1720
BPMF	0.2524	0.2667	0.2785	0.3031	0.3141
FastText-MF	0.3183	0.3390	0.3423	0.3532	0.3583
FastText-SAMF	0.3238	0.3396	0.3452	0.3569	0.3632
AMF+	0.3232	0.3391	0.3447	0.3564	0.3623
SAMF	**0.3242**	**0.3400**	**0.3454**	**0.3571**	**0.3633**

TABLE VI
NDCG COMPARISON AT K = 3

	NDCG@3(Train dataset)				
	#1	#2	#3	#4	#5
PMF	0.1736	0.1510	0.1511	0.1481	0.1453
BPMF	0.2140	0.2237	0.2363	0.2654	0.2795
FastText-MF	0.2918	0.3088	0.3136	0.3232	0.3253
FastText-SAMF	0.3015	0.3131	0.3176	0.3289	0.3345
AMF+	0.2999	0.3125	0.3173	0.3287	0.3340
SAMF	**0.3001**	**0.3139**	**0.3181**	**0.3302**	**0.3346**

TABLE VII
NDCG COMPARISON AT K = 1

	NDCG@1(Train dataset)				
	#1	#2	#3	#4	#5
PMF	0.1036	0.1016	0.1240	0.1313	0.1322
BPMF	0.1282	0.1452	0.1666	0.1935	0.2192
FastText-MF	0.2236	0.2457	0.2569	0.2724	0.2861
FastText-SAMF	0.2349	0.2535	0.2642	0.2821	0.2933
AMF+	0.2297	0.2507	0.2642	0.2821	0.2933
SAMF	**0.2369**	**0.2552**	**0.2642**	**0.2821**	**0.2933**

TABLE VIII
MAP COMPARISON IN MINI-DATASET-1

	MAP@ K(mini-dataset-1)				
	1	3	5	7	10
PMF	0.2209	0.2703	0.2827	0.2979	0.3027
BPMF	0.2384	0.2694	0.2803	0.2967	0.3085
FastText-MF	0.2558	0.2849	0.3001	0.2992	0.3035
FastText-SAMF	0.2471	0.2752	0.2838	0.3000	0.3009
AMF+	0.2413	0.2723	0.2937	0.3017	0.2995
SAMF	**0.2442**	**0.2703**	**0.2896**	**0.3067**	**0.3044**

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:02:02 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
MAP COMPARISON IN MINI-DATASET-2

	MAP@ K(mini-dataset-2)				
	1	3	5	7	10
PMF	0.2814	0.2744	0.2760	0.2794	0.2806
BPMF	0.3262	0.2757	0.2759	0.2792	0.2801
FastText-MF	0.3328	0.2754	0.2762	0.2805	0.2810
FastText-SAMF	0.3325	0.2755	0.2769	0.2801	0.2809
AMF+	0.3322	0.2758	0.2766	0.2796	0.2804
SAMF	**0.3322**	**0.2759**	**0.2771**	**0.2803**	**0.2809**

TABLE X
MAP COMPARISON IN MINI-DATASET-3

	MAP@ K(mini-dataset-3)				
	1	3	5	7	10
PMF	0.2788	0.2714	0.2725	0.2757	0.2773
BPMF	0.3206	0.2703	0.2727	0.2754	0.2772
FastText-MF	0.3359	0.2711	0.2726	0.2753	0.2770
FastText-SAMF	0.3256	0.2722	0.2727	0.2762	0.2770
AMF+	0.3256	0.2711	0.2724	0.2754	0.2771
SAMF	**0.3259**	**0.2714**	**0.2725**	**0.2760**	**0.2771**

TABLE XI
MAP COMPARISON IN MINI-DATASET-4

	MAP@ K(mini-dataset-4)				
	1	3	5	7	10
PMF	0.2727	0.2547	0.2566	0.2597	0.2596
BPMF	0.2984	0.2537	0.2565	0.2590	0.2594
FastText-MF	0.3029	0.2531	0.2565	0.2602	0.2598
FastText-SAMF	0.3025	0.2550	0.2565	0.2602	0.2596
AMF+	0.3025	0.2545	0.2567	0.2596	0.2595
SAMF	**0.3027**	**0.2552**	**0.2565**	**0.2603**	**0.2596**

VII, VIII, IX, X, and XI, respectively, and the following results can be observed:

According to Tables III, IV, V, VI, VII, and VII and Fig. 4 show that our method generally produces extremely good NDCG when compared to the baseline. More specifically, according to Fig. 4 we can observe that PMF and BPMF have the lowest NDCG, but BPMF performs better in comparison because of its better scalability. FastText-MF, FastText-SAMF and AMF+ perform better because they exploit contextual information.In the majority of situations, FastText-SAMF outperforms all other comparison models, demonstrating the efficacy of our proposed SAMF model. In addition, the NDCG score of our proposed SAMF model is either slightly higher than or on par with that of FastText-SAMF. This confirms that document input is not less effective than keyword input, in addition to being more convenient for developers. Although our method is not so superior to its counterparts, SAMF allows mashup developers to use documentation to express their individual development needs in order to best meet the developer's needs. This makes it easier for mashup developers without expertise to describe their needs. Additionally, we employ Doc2Vec technology to mine requirements documents, mashup descriptions, and API descriptions, as well as to mine full-text semantics by extracting complete events as opposed to extracting keywords. This can guarantee that the mined information preserves the semantic link to the fullest extent possible and more properly matches the developer's specific requirements. In order to overcome the problem of sparse data, we use the obtained context information as auxiliary information in the model, and introduce adversarial training to add noise to the data to further solve this problem and improve the generalization robustness of the model.

Notably, the NDCG of most models does not decrease with increasing mask data size; only the PMF decreases slightly. More actual data will be present in the test set if the size of the mask data is increased. When computing the NDCG, it is natural that the DCG would increase according to the quantity of real data.

When comparing MAP, we constructed 4 mini-datasets, which were selected from the previous Train dataset #1 to #5. In mini-dataset-1 (a mashup consists of more than 1 API), we randomly picked one API invocation per mashup for the test set, and the remaining data for the train set. In mini-dataset-2 (a mashup consists of more than two APIs), we randomly choose two API calls from each mashup for the test set, while the other data is placed in the train set. In mini-dataset-3 (a mashup consists of more than three APIs) and mini-dataset-4 (a mashup consists of more than four APIs), the data is processed in the same way as in mini-dataset-2. Since the experimental results are similar across different datasets, we only show graphs of the MAP scores in the mini-datasets obtained from the Train dataset #5.

From the four subgraphs of Fig. 5, we can notice that when $K > 7$, the MAP score of Fig. 5(a) rises abruptly and then gradually becomes flat. We do not punish MAP for incorrect suggestions when adding new APIs to the recommendation list, thus the MAP scores of the other three subgraphs decrease rapidly from $K = 1$ to $K = 3$, and then climb steadily. Therefore, the accuracy of the first three APIs suggested to developers is greatest. Whenever the number of suggested APIs rises, MAP performance will degrade progressively. Overall, our proposed method is generally no worse than all baselines in all scenarios in terms of MAP in the top-10 recommendations, indicating that our model is more accurate in specifying and ranking relevant APIs for more efficient recommendations.

From Fig. 5, it is also observed that PMF and BPMF have the weakest performance. This demonstrates that the generalization ability of the pure PMF model is insufficient, and that using the BPMF model alone is not sufficient to exploit the contextual information between mashup and API. FastText-MF, FastText-SAMF and AMF+ perform better because they leverage contextual information between the mashup and the API. Among all the baselines, SAMF has the best performance. This suggests that leveraging contextual information and the introduction of adversarial training can improve API recommendations for mashups to a certain extent.

2) Impact of Framework Components (RQ 2): To answer RQ2, we investigate a dual strategy of text similarity and adversarial training. In particular, we examine the consequences of these tactics by defining three variations of SAMF:

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:02:02 UTC from IEEE Xplore. Restrictions apply.

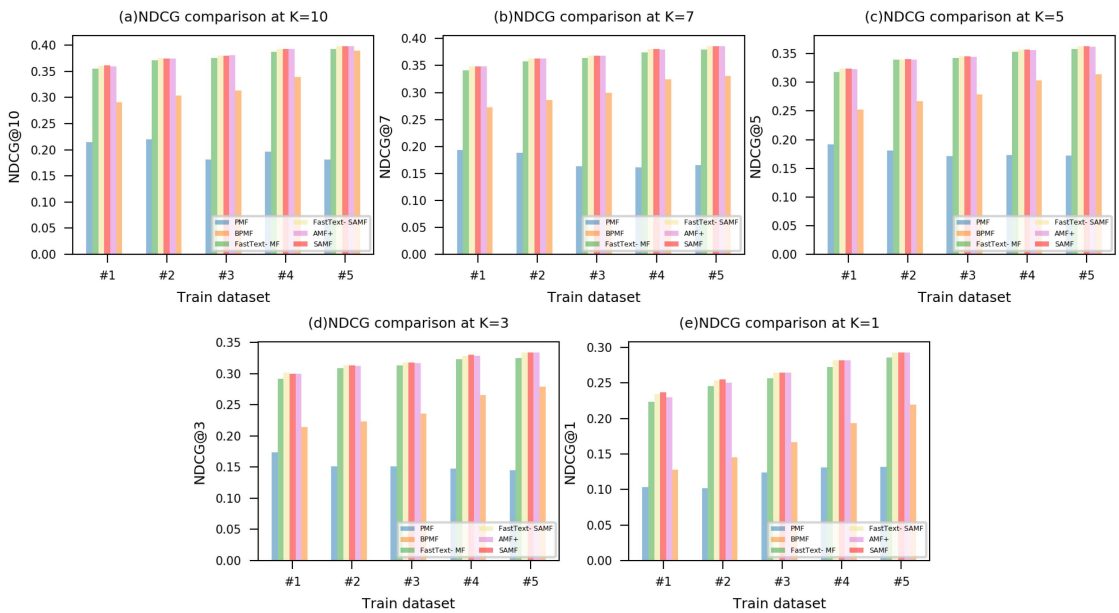

Fig. 4. NDCG comparison.

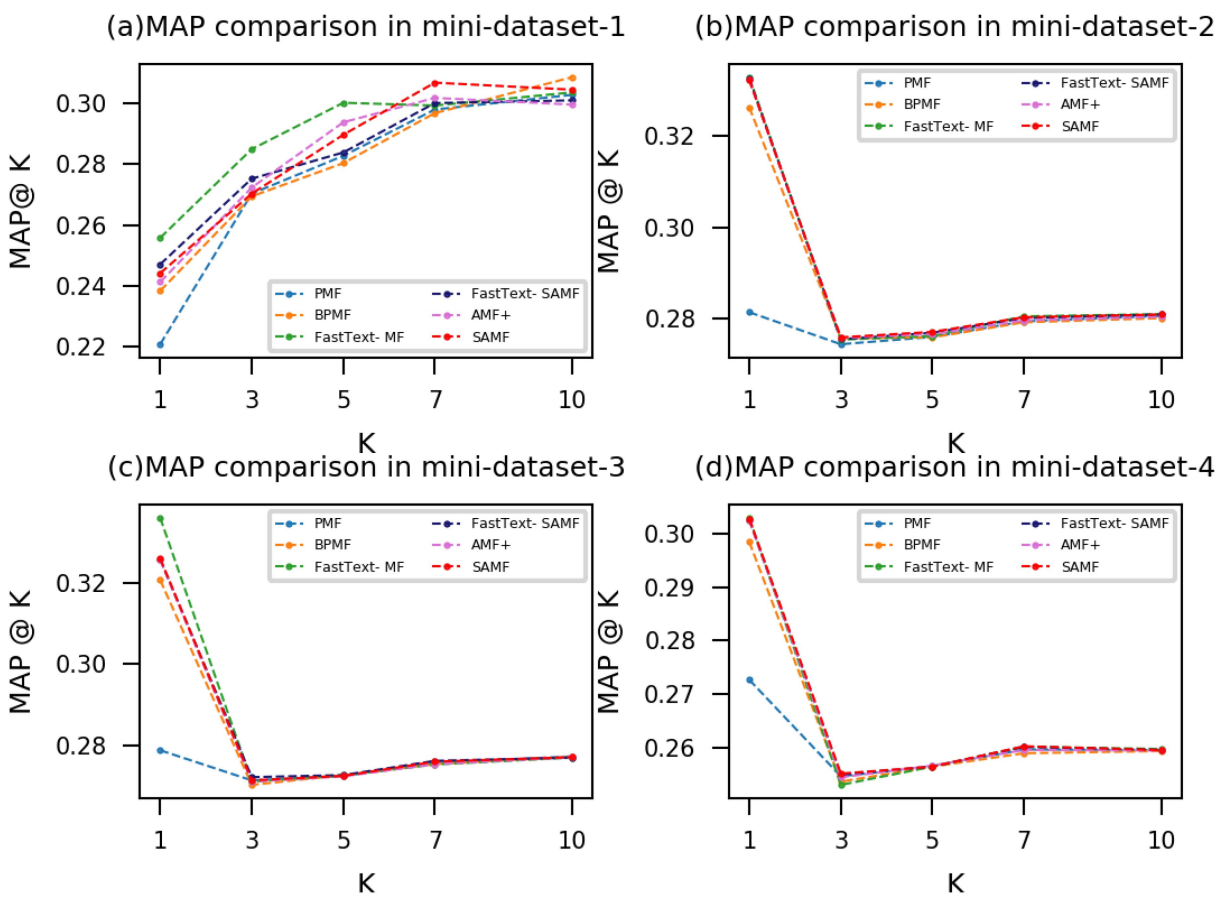

Fig. 5. MAP comparison.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:02:02 UTC from IEEE Xplore. Restrictions apply.

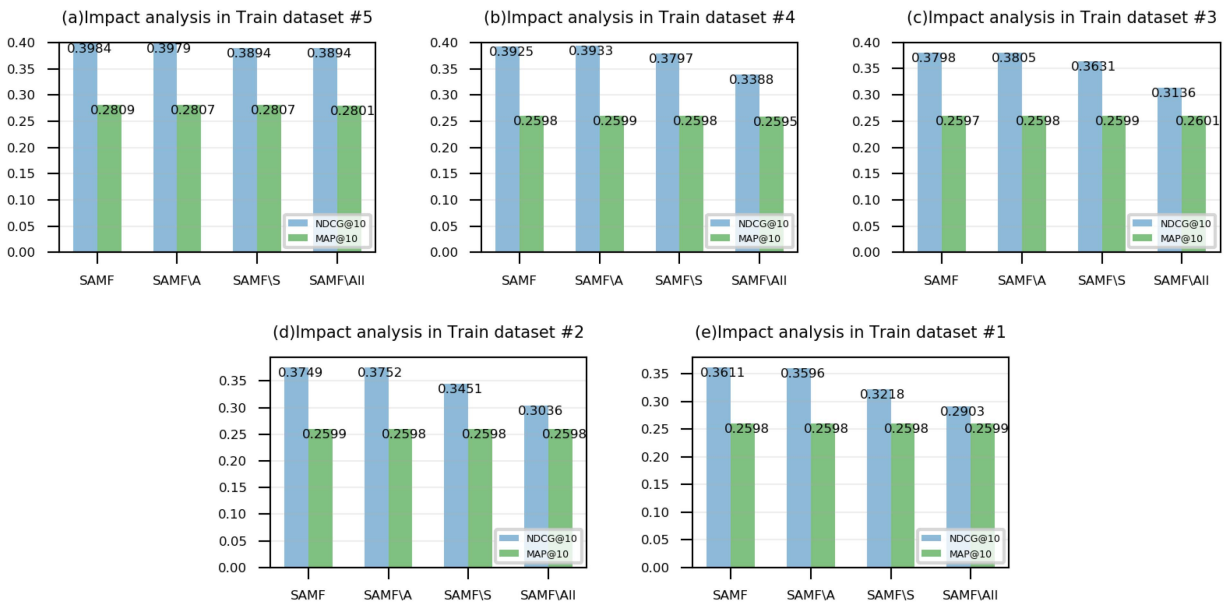

Fig. 6. Impact analysis of SAMF framework components.

TABLE XII
NDCG COMPARISON AT K = 10

	NDCG@10(Train dataset)				
	#1	#2	#3	#4	#5
SAMF\All	0.2903	0.3036	0.3136	0.3388	0.3894
SAMF\A	0.3596	0.3752	0.3805	0.3933	0.3979
SAMF\S	0.3218	0.3451	0.3631	0.3797	0.3894
SAMF	**0.3611**	**0.3749**	**0.3798**	**0.3925**	**0.3984**

TABLE XIII
NDCG COMPARISON AT K = 7

	NDCG@7(Train dataset)				
	#1	#2	#3	#4	#5
SAMF\	0.2728	0.2865	0.3000	0.3247	0.3307
SAMF\	0.3488	0.3631	0.3690	0.3805	0.3864
SAMF\	0.3099	0.3305	0.3500	0.3668	0.3759
SAMF	**0.3482**	**0.3630**	**0.3686**	**0.3805**	**0.3863**

-SAMF\A: SAMF without adversarial training. During training, it eliminates adversarial training and retains the remainder.

-SAMF\S: SAMF without text similarity. During the training process, the part of adding text similarity as auxiliary information into the model is removed.

-SAMF\All: SAMF without text similarity and adversarial training. It uses a pure BPMF model for training. After extensive experiments, the best performance of the three variants is shown in Fig. 6, and their detailed data are in Tables XII, XIII, XIV, XV, XVI, XVII, XVIII, XIX, and XX (Since the experimental results are similar across different datasets, we only show graphs of the MAP scores in the mini-datasets obtained from the Train dataset #5), which we observed:

TABLE XIV
NDCG COMPARISON AT K = 5

	NDCG@5(Train dataset)				
	#1	#2	#3	#4	#5
SAMF\All	0.2524	0.2667	0.2785	0.3031	0.3141
SAMF\A	0.3244	0.3400	0.3458	0.3572	0.3632
SAMF\S	0.2832	0.3106	0.3290	0.3451	0.3544
SAMF	**0.3242**	**0.3400**	**0.3454**	**0.3571**	**0.3633**

TABLE XV
NDCG COMPARISON AT K = 3

	NDCG@3(Train dataset)				
	#1	#2	#3	#4	#5
SAMF\All	0.2140	0.2237	0.2363	0.2654	0.2795
SAMF\A	0.2988	0.3134	0.3184	0.3299	0.3352
SAMF\S	0.2475	0.2725	0.2976	0.3122	0.3237
SAMF	**0.3001**	**0.3139**	**0.3181**	**0.3302**	**0.3346**

TABLE XVI
NDCG COMPARISON AT K = 1

	NDCG@1(Train dataset)				
	#1	#2	#3	#4	#5
SAMF\All	0.1282	0.1452	0.1666	0.1935	0.2192
SAMF\A	0.2318	0.2507	0.2646	0.2821	0.2933
SAMF\S	0.1651	0.1893	0.2316	0.2527	0.2756
SAMF	**0.2369**	**0.2552**	**0.2642**	**0.2821**	**0.2933**

– If there is no adversarial training part, the performance will drop. Taking the score of NDCG@1 on five datasets as an example, it is reduced by 2.152% on Train dataset #1, 1.763% on Train dataset #2, and unchanged on the remaining three datasets. MAP@1 hardly changes on the four datasets. This shows that

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:02:02 UTC from IEEE Xplore. Restrictions apply.

TABLE XVII
MAP COMPARISON IN MINI-DATASET-1

	MAP@ K(mini-dataset-1)				
	1	3	5	7	10
SAMF\All	0.2384	0.2694	0.2803	0.2967	0.3085
SAMF\A	0.2442	0.2733	0.2716	0.3038	0.3082
SAMF\S	0.2471	0.2791	0.2914	0.2996	0.3053
SAMF	**0.2442**	**0.2703**	**0.2896**	**0.3067**	**0.3044**

TABLE XVIII
MAP COMPARISON IN MINI-DATASET-2

	MAP@ K(mini-dataset-2)				
	1	3	5	7	10
SAMF\All	0.3262	0.2757	0.2759	0.2792	0.2801
SAMF\A	0.3322	0.2757	0.2763	0.2800	0.2807
SAMF\S	0.3319	0.2755	0.2764	0.2795	0.2807
SAMF	**0.3322**	**0.2759**	**0.2771**	**0.2803**	**0.2809**

TABLE XIX
MAP COMPARISON IN MINI-DATASET-3

	MAP@ K(mini-dataset-3)				
	1	3	5	7	10
SAMF\All	0.3206	0.2703	0.2727	0.2754	0.2772
SAMF\A	0.3259	0.2707	0.2727	0.2760	0.2774
SAMF\S	0.3262	0.2703	0.2728	0.2756	0.2772
SAMF	**0.3259**	**0.2714**	**0.2725**	**0.2760**	**0.2771**

TABLE XX
MAP COMPARISON IN MINI-DATASET-4

	MAP@ K(mini-dataset-4)				
	1	3	5	7	10
SAMF\All	0.2984	0.2537	0.2565	0.2590	0.2594
SAMF\A	0.3027	0.2527	0.2562	0.2603	0.2598
SAMF\S	0.3027	0.2537	0.2565	0.2601	0.2600
SAMF	**0.3027**	**0.2552**	**0.2565**	**0.2603**	**0.2596**

adversarial training helps improve model accuracy, but the effect is not significant. As the masking percentage becomes larger and larger, the drop rate becomes smaller, but the gap is not large, which proves that adversarial training in this model has little capacity to address the data sparsity issue.

– Without the auxiliary information of text similarity, NDCG@1 has a drop rate of 30.31%, 25.82%, 12.34%, 10.42%, and 6.03% on the five datasets Train dataset #1 to Train dataset #5. MAP@1 hardly changes on the four datasets. It shows that it is effective to introduce auxiliary information about text similarity into the model. As the masking percentage becomes larger and larger, the drop rate becomes smaller and smaller, which proves that contextual information as auxiliary information in the model can indeed alleviate the data sparsity problem to a large extent.

– Without auxiliary information and adversarial training, the performance drops the most. The drop rate of NDCG@1 on five datasets is 25.26%–45.88%. The drop rate of MAP@1 on the four datasets is 1.42%–2.38%. This highlights the significance and usefulness of the suggested SAMF framework.

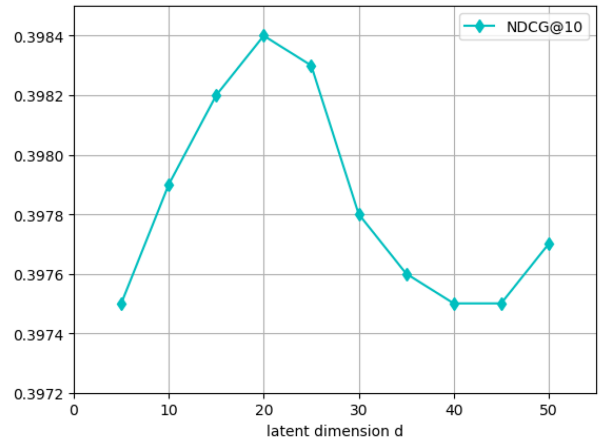

Fig. 7. Impact analysis of latent dimension d.

Similar to the previous pattern, the drop rate of NDCG@K in the three trials will decrease as K increases, but the decline rate of MAP@K will grow as K increases. MAP not only considers the accuracy of the recommendation list, but also considers whether the recommendation results are in front, so the smaller the recommendation list, the less affected the MAP. From the experimental data in the above graphs, it can be observed that our proposed SAMF is effective.

3) Hyperparameters Analysis (RQ 3): This part discusses the effect of model hyperparameters used for data training on the performance of recommendations. To perform experiments, we selected the Train dataset #5 dataset, fixed all other parameters, and altered just the hyperparameters. The hyperparameters consist of the mashup or API latent dimension d, the learning rate lr, and the regularization parameter λ_S for auxiliary information on the model during training.

The influence of the feature dimension of mashup and API on NDCG@10 for the Train data #5 dataset is shown in Fig. 7. Observably, increasing the feature dimension of the mashup and API enhances the performance of recommendations. When the feature dimension is increased from 5 to 20, NDCG@10 rises from 0.3975 to 0.3984. However, as seen in the graph, as the feature dimension approaches 20, the NDCG@10 value begins to quickly decrease. This discovery shows that a modest feature dimension may offer enough storage capacity for information during training. Some mashup or API data in the feature dimension may be lost if the feature dimension is too tiny. In contrast, if the feature size is excessively big, it may lead to duplication and raise the model training's computational time overhead.

To verify that the trials are comparable, we employ the control variable approach to examine the impacts of various learning rates on the model training effect. As shown in Fig. 8, the API suggestion produces the greatest results when the learning rate is 0.00001. When the learning rate rises owing to a high learning rate, the model learns the data more quickly and reaches the fitting point earlier, which may cause the parameters to fluctuate on both sides of the optimum solution and fail to attain the ideal value.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:02:02 UTC from IEEE Xplore. Restrictions apply.

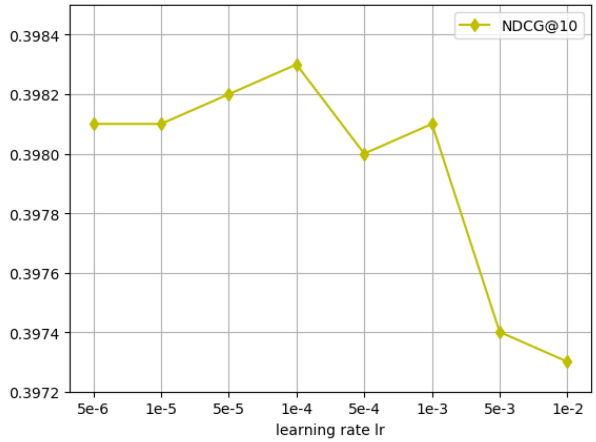

Fig. 8. Impact analysis of learning rate l_r.

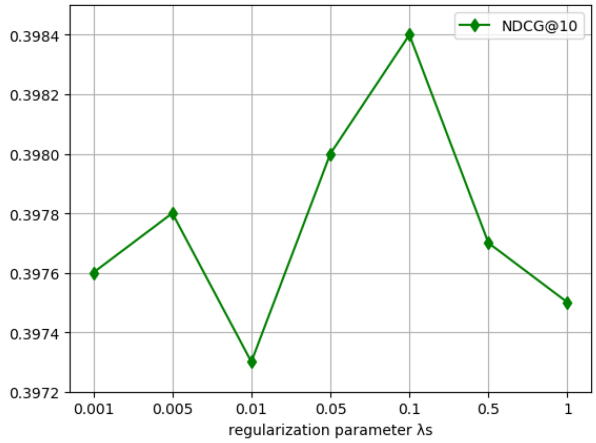

Fig. 9. Impact analysis of regularization parameter λ_S.

In addition to feature dimension and learning rate, the most crucial hyperparameter is the textual similarity of auxiliary information's regularization coefficient λ_S. Regularization involves the addition of a regularization term (penalty term) after the loss function. In reality, it is sometimes referred to as a structural risk minimization method, consisting of empirical risk (loss function) and regularization to minimize model overfitting. Here, we will investigate the effect of λ_S on model performance. As seen in Fig. 9, NDCG@10 achieves its ideal value when λ_S is set to 0.1. However, when λ_S is more than 0.1, the performance of model suggestion quickly declines, indicating that excessive regularization would negatively impact the training of the model.

V. Conclusion

Aiming at the problem of developer demand mining and the sparse historical data of mashups and APIs, this paper fully mines the context information of development requirements, functional descriptions of mashups and APIs through full-text semantic mining, and utilizes collaborative filtering and adversarial training to alleviate the data sparse problem. Specifically, by establishing a SAMF model, in which Dov2Vec technology is used to perform full-text semantic mining on developer requirements, mashup function description and API function description to obtain a document matrix, and the cosine function is used to calculate the text similarity between the three. Further, use BPMF to mine information from the sparse historical data of developer requirements, mashups, and APIs, and incorporate the obtained context information as auxiliary information into model training to relieve the data sparsity issue. In addition, adversarial training is introduced to further solve the data sparsity issue and increase the model's robustness. This paper also investigates the impact of contextual information and adversarial training on predictions. Taking the score of NDCG@10 as an example, if the context information is injected into the model, the score of NDCG@10 improves by 2.18%-23.87% on each dataset; if the adversarial training is injected into the model, the score improves by 0-15.78%; if both are injected into the model at the same time, the score improves by 0-24.39%. The scores of other evaluation metrics are also improved to varying degrees, which verifies the effectiveness of injecting contextual information and adversarial training into the model.

This paper focuses on development requirements and functional descriptions of mashups and APIs, while ignoring other valuable information such as API QoS (such as version update time) and compatibility between APIs. For our future work, we will further mine the information of other important APIs and mashups contained in the dataset to improve the SAMF model, so that the recommended APIs are more appropriate versions and compatible with each other. For instance, certain APIs may be frequently combined with other APIs, or certain API combinations may be more useful for specific sorts of mashups. In the future, we will also investigate adding user feedback to the process of generating recommendations in order to discover the preferences of developers and obtain more personalized recommendations. In addition, we are contemplating integrating human evaluation ratings in the future in order to measure the recommendation outcomes more accurately.

References

[1] S. Badsha et al., "Privacy preserving location-aware personalized web service recommendations," *IEEE Trans. Serv. Comput.*, vol. 14, no. 3, pp. 791–804, May/Jun. 2021.

[2] X. Fan, Y. Hu, Z. Zheng, Y. Wang, P. Brézillon, and W. Chen, "CASR-TSE: Context-aware web services recommendation for modeling weighted temporal-spatial effectiveness," *IEEE Trans. Serv. Comput.*, vol. 14, no. 1, pp. 58–70, Jan./Feb. 2021.

[3] J. Yu, B. Benatallah, F. Casati, and F. Daniel, "Understanding mashup development," *IEEE Internet Comput.*, vol. 12, no. 5, pp. 44–52, Sep./Oct. 2008.

[4] H. Zhao, J. Wang, Q. Zhou, X. Wang, and H. Wu, "Web API recommendation with features ensemble and learning-to-rank," in *Proc. CCF Conf. Big Data*, 2019, pp. 406–419.

[5] B. Cao, X. F. Liu, M. M. Rahman, B. Li, J. Liu, and M. Tang, "Integrated content and network-based service clustering and web APIs recommendation for mashup development," *IEEE Trans. Serv. Comput.*, vol. 13, no. 1, pp. 99–113, Jan./Feb. 2021.

[6] X. Wang, X. Liu, J. Liu, X. Chen, and H. Wu, "A novel knowledge graph embedding based API recommendation method for mashup development," *World Wide Web*, vol. 24, no. 3, pp. 869–894, 2021.

[7] L. Qi, W. Lin, X. Zhang, W. Dou, X. Xu, and J. Chen, "A correlation graph based approach for personalized and compatible web APIs recommendation in mobile app development," *IEEE Trans. Knowl. Data Eng.*, early access, Apr. 26, 2022, doi: 10.1109/TKDE.2022.3168611.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:02:02 UTC from IEEE Xplore. Restrictions apply.

[8] B. Tang, M. Tang, Y. Xia, and M.-Y. Hsieh, "Composition pattern-aware web service recommendation based on depth factorisation machine," *Connection Sci.*, vol. 33, no. 4, pp. 870–890, 2021.
[9] B. Xia, Y. Fan, W. Tan, K. Huang, J. Zhang, and C. Wu, "Category-aware API clustering and distributed recommendation for automatic mashup creation," *IEEE Trans. Serv. Comput.*, vol. 8, no. 5, pp. 674–687, Sep./Oct. 2015.
[10] M. Shi, J. Liu, D. Zhou, M. Tang, F. Xie, and T. Zhang, "A probabilistic topic model for mashup tag recommendation," in *Proc. IEEE Int. Conf. Web Serv.*, 2016, pp. 444–451.
[11] L. Qi et al., "Finding all you need: Web APIs recommendation in web of things through keywords search," *IEEE Trans. Comput. Social Syst.*, vol. 6, no. 5, pp. 1063–1072, Oct. 2019.
[12] C. Li, R. Zhang, J. Huai, and H. Sun, "A novel approach for API recommendation in mashup development," in *Proc. IEEE Int. Conf. Web Serv.*, 2014, pp. 289–296.
[13] H. Li, J. Liu, B. Cao, M. Tang, X. Liu, and B. Li, "Integrating tag, topic, co-occurrence, and popularity to recommend web APIs for mashup creation," in *Proc. IEEE Int. Conf. Serv. Comput.*, 2017, pp. 84–91.
[14] K. K. Fletcher, "A quality-aware web API recommender system for mashup development," in *Proc. Int. Conf. Serv. Comput.*, 2019, pp. 1–15.
[15] Q. Gu, J. Cao, and Q. Peng, "Service package recommendation for mashup creation via mashup textual description mining," in *Proc. IEEE Int. Conf. Web Serv.*, 2016, pp. 452–459.
[16] N. Zhang, J. Wang, and Y. Ma, "Mining domain knowledge on service goals from textual service descriptions," *IEEE Trans. Serv. Comput.*, vol. 13, no. 3, pp. 488–502, May/Jun. 2017.
[17] C. H. Mendhe, N. Henderson, G. Srivastava, and V. Mago, "A scalable platform to collect, store, visualize, and analyze Big Data in real time," *IEEE Trans. Comput. Social Syst.*, vol. 8, no. 1, pp. 260–269, Feb. 2020.
[18] C. Li, R. Zhang, J. Huai, X. Guo, and H. Sun, "A probabilistic approach for web service discovery," in *Proc. IEEE Int. Conf. Serv. Comput.*, 2013, pp. 49–56.
[19] B. Cao, X. F. Liu, J. Liu, and M. Tang, "Domain-aware mashup service clustering based on LDA topic model from multiple data sources," *Inf. Softw. Technol.*, vol. 90, pp. 40–54, 2017.
[20] L. Yao, X. Wang, Q. Z. Sheng, B. Benatallah, and C. Huang, "Mashup recommendation by regularizing matrix factorization with API co-invocations," *IEEE Trans. Serv. Comput.*, vol. 14, no. 2, pp. 502–515, Mar./Apr. 2021.
[21] L. Qi, H. Song, X. Zhang, G. Srivastava, X. Xu, and S. Yu, "Compatibility-aware web API recommendation for mashup creation via textual description mining," *ACM Trans. Multimidia Comput. Commun. Appl.*, vol. 17, no. 1 s, pp. 1–19, 2021.
[22] M. Nguyen, J. Yu, T. Nguyen, and Y. Han, "Attentional matrix factorization with context and co-invocation for service recommendation," *Expert Syst. Appl.*, vol. 186, 2021, Art. no. 115698.
[23] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *Proc. 10th Int. Conf. World Wide Web*, 2001, pp. 285–295.
[24] G. Linden, B. Smith, and J. York, "Amazon. com recommendations: Item-to-item collaborative filtering," *IEEE Internet Comput.*, vol. 7, no. 1, pp. 76–80, Jan./Feb. 2003.
[25] J. L. Herlocker, J. A. Konstan, A. Borchers, and J. Riedl, "An algorithmic framework for performing collaborative filtering," in *Proc. 22nd Annu. Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 1999, pp. 230–237.
[26] J. Zhong and X. Li, "Unified collaborative filtering model based on combination of latent features," *Expert Syst. Appl.*, vol. 37, no. 8, pp. 5666–5672, 2010.
[27] J. Bobadilla, F. Ortega, A. Hernando, and A. Gutiérrez, "Recommender systems survey," *Knowl.-Based Syst.*, vol. 46, pp. 109–132, 2013.
[28] M. Tang, Z. Zheng, G. Kang, J. Liu, Y. Yang, and T. Zhang, "Collaborative web service quality prediction via exploiting matrix factorization and network map," *IEEE Trans. Netw. Service Manag.*, vol. 13, no. 1, pp. 126–137, Mar. 2016.
[29] Y. Zhang, C. Yin, Q. Wu, Q. He, and H. Zhu, "Location-aware deep collaborative filtering for service recommendation," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 51, no. 6, pp. 3796–3807, Jun. 2021.
[30] W. Liang et al., "Deep neural network security collaborative filtering scheme for service recommendation in intelligent cyber-physical systems," *IEEE Internet Things J.*, vol. 9, no. 22, pp. 22123–22132, Nov. 2021.
[31] R. Salakhutdinov, "Probabilistic matrix factorization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2008, pp. 1257–1264.
[32] L. Chen, Y. Feng, J. Wu, and Z. Zheng, "An enhanced QoS prediction approach for service selection," in *Proc. IEEE Int. Conf. Serv. Comput.*, 2011, pp. 727–728.
[33] P. He, J. Zhu, Z. Zheng, J. Xu, and M. R. Lyu, "Location-based hierarchical matrix factorization for web service recommendation," in *Proc. IEEE Int. Conf. web Serv.*, 2014, pp. 297–304.
[34] S. Li, J. Wen, F. Luo, T. Cheng, and Q. Xiong, "A location and reputation aware matrix factorization approach for personalized quality of service prediction," in *Proc. IEEE Int. Conf. Web Serv.*, 2017, pp. 652–659.
[35] H. Wu, K. Yue, B. Li, B. Zhang, and C.-H. Hsu, "Collaborative QoS prediction with context-sensitive matrix factorization," *Future Gener. Comput. Syst.*, vol. 82, pp. 669–678, 2018.
[36] R. Salakhutdinov and A. Mnih, "Bayesian probabilistic matrix factorization using markov chain monte carlo," in *Proc. 25th Int. Conf. Mach. Learn.*, 2008, pp. 880–887.
[37] C. Yan, Q. Zhang, and X. Zhao, "A method of Bayesian probabilistic matrix factorization based on generalized Gaussian distribution," *J. Comput. Res. Develop.*, vol. 53, no. 12, pp. 2793–2800, 2016.
[38] D. Kim, C. Park, J. Oh, and H. Yu, "Deep hybrid recommender systems via exploiting document context and statistics of items," *Inf. Sci.*, vol. 417, pp. 72–87, 2017.
[39] C. Musto, P. Lops, M. de Gemmis, and G. Semeraro, "Context-aware graph-based recommendations exploiting personalized pagerank," *Knowl.-Based Syst.*, vol. 216, 2021, Art. no. 106806.
[40] M. Kolahkaj, A. Harounabadi, A. Nikravanshalmani, and R. Chinipardaz, "Incorporating multidimensional information into dynamic recommendation process to cope with cold start and data sparsity problems," *J. Ambient Intell. Humanized Comput.*, vol. 12, no. 10, pp. 9535–9554, 2021.
[41] C. Martinez-Cruz, C. Porcel, J. Bernabé-Moreno, and E. Herrera-Viedma, "A model to represent users trust in recommender systems using ontologies and fuzzy linguistic modeling," *Inf. Sci.*, vol. 311, pp. 102–118, 2015.
[42] C. Park, D. Kim, J. Oh, and H. Yu, "Improving top-k recommendation with truster and trustee relationship in user trust network," *Inf. Sci.*, vol. 374, pp. 100–114, 2016.
[43] Y. Zhong, Y. Fan, W. Tan, and J. Zhang, "Web service recommendation with reconstructed profile from mashup descriptions," *IEEE Trans. Automat. Sci. Eng.*, vol. 15, no. 2, pp. 468–478, Apr. 2018.
[44] L. Yao, Q. Z. Sheng, A. H. Ngu, J. Yu, and A. Segev, "Unified collaborative and content-based web service recommendation," *IEEE Trans. Serv. Comput.*, vol. 8, no. 3, pp. 453–466, May/Jun. 2014.
[45] Z. Zheng, H. Ma, M. R. Lyu, and I. King, "QoS-aware web service recommendation by collaborative filtering," *IEEE Trans. Serv. Comput.*, vol. 4, no. 2, pp. 140–152, Jun. 2010.
[46] L. Shao, J. Zhang, Y. Wei, J. Zhao, B. Xie, and H. Mei, "Personalized QoS prediction forweb services via collaborative filtering," in *Proc. IEEE Int. Conf. Web Serv.*, 2007, pp. 439–446.
[47] L. Yao, X. Wang, Q. Z. Sheng, W. Ruan, and W. Zhang, "Service recommendation for mashup composition with implicit correlation regularization," in *Proc. IEEE Int. Conf. Web Serv.*, 2015, pp. 217–224.
[48] K. Fletcher, "Regularizing matrix factorization with implicit user preference embeddings for web API recommendation," in *Proc. IEEE Int. Conf. Serv. Comput.*, 2019, pp. 1–8.
[49] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in *Proc. 26th Int. Conf. World Wide Web*, 2017, pp. 173–182.
[50] D. Croft, S. Coupland, J. Shell, and S. Brown, "A fast and efficient semantic short text similarity metric," in *Proc. 13th U.K. Workshop Comput. Intell.*, 2013, pp. 221–227.
[51] R. Singh and S. Singh, "Text similarity measures in news articles by vector space model using NLP," *J. Inst. Engineers: Ser. B*, vol. 102, no. 2, pp. 329–338, 2021.
[52] R. M. Neal, "Probabilistic inference using markov chain monte carlo methods," Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, 1993.
[53] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, "Bag of tricks for efficient text classification," 2016, *arXiv:1607.01759*.
[54] Q. Le and T. Mikolov, "Distributed representations of sentences and documents," in *Proc. Int. Conf. Mach. Learn.*, 2014, pp. 1188–1196.

Chunyan Sang received the PhD degree in control theory and control engineering from Chongqing University, Chongqing, China, in 2014. She is currently an associate professor with the School of Software Engineering, Chongqing University of Posts and Telecommunications. Her research interests include service recommendation, data mining, and its applications.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:02:02 UTC from IEEE Xplore. Restrictions apply.

Xinyan Deng received the BS degree from the Chongqing University of Posts and Telecommunications, Chongqing, China, in 2021. She is currently working toward the MS degree with the School of Software Engineering, Chongqing University of Posts and Telecommunications. Her research interests include service discovery and recommendation.

Shigen Liao received the PhD degree in management science and engineering from Chongqing University, Chongqing, China, in 2021. He is currently an assistant engineer with the School of Software Engineering, Chongqing University of Posts and Telecommunications. His research interests include intelligent manufacturing, knowledge service, and quality management.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:02:02 UTC from IEEE Xplore. Restrictions apply.